Отчёт по проекту «LLM-AlertBridge: модуль интеграции SOC-событий с языковой моделью»

1. Состав проектной команды

Роль	Участник	Группа	Контакт
Руководитель проекта	Вишнеков Андрей Владленович	—	[указать e-mail]
Инженер-программист (Python)	Полищук Максим Вячеславович	МКС242	[указать e-mail]
Аналитик по обработке данных и тестированию	Ковригина Софья Витальевна	МКС242	[указать e-mail]

Заказчик: МИЭМ / ДКИ, НИУ ВШЭ.

---

2. Актуальность проекта

Центры мониторинга информационной безопасности (SOC) ежедневно обрабатывают тысячи событий, поступающих из SIEM-систем. Значительная доля этих событий составляет ложные срабатывания (false positives), что приводит к перегрузке аналитиков и замедлению реакции на реальные инциденты. По оценкам отрасли, от 40 до 70 % рабочего времени аналитика первой линии уходит на рутинный триаж событий — первичную классификацию, сбор контекста и принятие решения о необходимости дальнейшего расследования.

Одновременно развитие больших языковых моделей (LLM) открывает возможности для автоматизации начального этапа анализа инцидентов. Однако применение облачных LLM-сервисов в контексте SOC ограничено требованиями конфиденциальности: данные событий могут содержать чувствительную информацию об инфраструктуре организации (IP-адреса, имена хостов, учётные записи, индикаторы уязвимостей). Отправка таких данных на внешние серверы нарушает базовые принципы информационной безопасности.

В связи с этим возникает задача создания автономного модуля, который интегрируется с существующей SIEM-инфраструктурой и использует локально развёрнутую LLM-модель для семантического анализа событий без передачи данных за периметр контролируемой инфраструктуры.

---

3. Цели, задачи, планируемый и фактический результат

3.1. Цель проекта

Разработка и исследование прототипа программного модуля, интегрирующего систему оповещений SIEM с локальной LLM-моделью для анализа и оценки инцидентов информационной безопасности, с

последующей экспериментальной оценкой его эффективности по снижению нагрузки на аналитиков SOC.

3.2. Задачи

1. Проектирование архитектуры модуля, обеспечивающей интеграцию SIEM (Wazuh) с локально развёрнутой LLM через защищённый канал связи.
2. Реализация серверной части (backend) для приёма, нормализации и хранения событий из Wazuh.
3. Разработка механизма формирования структурированных запросов (промптов) к языковой модели и парсинга её ответов.
4. Создание клиента интеграции с LM Studio, обеспечивающего OpenAI-совместимое взаимодействие с локальной LLM.
5. Развёртывание тестовой инфраструктуры в облачной среде (Yandex Cloud) с SIEM-агентами, имитацией атак и VPN-доступом.
6. Разработка веб-интерфейса для визуализации событий и результатов LLM-анализа.
7. Сравнительный анализ локальных LLM-моделей для задачи анализа событий SOC.
8. Подготовка технической документации.

3.3. Планируемый результат

Исследовательский прототип программного модуля (уровень TRL 4–5), включающий:

- модуль приёма событий из SIEM;
- механизм нормализации и хранения данных;
- механизм безопасной передачи данных в локальную LLM;
- шаблоны запросов к модели и парсер структурированных ответов;
- веб-интерфейс для работы аналитика;
- техническую документацию и инструкцию по развёртыванию.

3.4. Фактический результат

На текущем этапе реализован функциональный прототип, включающий все заявленные компоненты первой стадии проекта:

Компонент	Статус	Описание
Интеграция с Wazuh	Реализован	Асинхронный клиент для Wazuh Indexer (OpenSearch); получение и дедупликация событий
Нормализация событий	Реализован	Преобразование сырых данных Wazuh в унифицированный внутренний формат
Хранение данных	Реализован	PostgreSQL 16 с JSONB-столбцами, SQLAlchemy 2.0 async, Alembic-миграции
Интеграция с LLM	Реализован	Клиент LM Studio через OpenAI SDK, шаблоны промптов, парсинг JSON-ответов
Оркестрация анализа	Реализован	Конвейер «событие → промпт → LLM → парсинг → сохранение» с повторными попытками

Компонент	Статус	Описание
Веб-интерфейс	Реализован	Jinja2 + HTMX + Tailwind CSS; список событий, детальная страница, запуск анализа
REST API	Реализован	FastAPI; эндпоинты CRUD, синхронизации, анализа, health-check
Инфраструктура	Реализован	Docker Compose (PostgreSQL + Wazuh 4.9.2 + backend); Yandex Cloud, OpenVPN
Документация	Реализован	Описание архитектуры, сети, развёртывания; README с инструкциями
Сравнение моделей	Реализован	Бенчмарки пяти LLM по общим и домен-специфичным метрикам
Контекстное обогащение (osquery)	Не реализован	Заявлен как направление дальнейшего развития
Автоматизированные тесты	Не реализован	Инфраструктура pytest подготовлена, тестовые случаи отсутствуют

4. Новизна и преимущества полученных решений

4.1. Архитектурное решение с локальной LLM

В отличие от существующих подходов, использующих облачные LLM-сервисы (OpenAI API, Google Gemini и др.), разработанный модуль принципиально основан на локальном выполнении языковой модели. Модель развёрнута на рабочей станции аналитика через LM Studio, а связь с облачным бэкендом осуществляется через зашифрованный VPN-туннель. Это исключает передачу чувствительных данных об инфраструктуре за пределы контролируемой среды. В случае с реальной разверткой в рамках инфраструктуры компании модель LLM можно поднять на более мощном сервере, находящимся во внутреннем контуре компании, что точно так же обеспечит конфиденциальность данных, которые в неё попадают, а также позволит быстрее проводить анализ событий.

4.2. Структурированный формат анализа

Система формирует структурированный JSON-ответ, включающий шесть фиксированных полей: краткое описание события, гипотезу о природе инцидента, возможные причины, ключевые индикаторы, рекомендуемые проверки и оценку уровня уверенности модели. Такой формат упрощает интеграцию результатов анализа в рабочие процессы SOC и обеспечивает единообразие представления данных.

4.3. Нормализация и сохранение контекста

Реализован механизм нормализации сырых данных Wazuh в унифицированный формат с извлечением ключевых полей (идентификатор правила, описание, уровень критичности, информация об агенте, сетевые и пользовательские данные). Как исходные, так и нормализованные

данные сохраняются в JSONB-столбцах PostgreSQL, что обеспечивает возможность повторного анализа и ретроспективного исследования.

4.4. Роль модели как ассистента

Языковая модель выступает исключительно как интеллектуальный помощник аналитика. Система не принимает автономных решений и не инициирует автоматического реагирования. Окончательное суждение всегда остаётся за человеком-оператором.

---

5. Степень обоснованности и достоверности полученных результатов

5.1. Обоснованность архитектурных решений

Выбор каждого технологического компонента документирован и обоснован:

- **Wazuh** — открытая SIEM-система с активным сообществом, богатой библиотекой правил и поддержкой MITRE ATT&CK.
- **FastAPI** — асинхронный Python-фреймворк с автогенерацией OpenAPI-документации и встроенной поддержкой Pydantic-моделей для строгой валидации данных.
- **PostgreSQL 16** — поддержка JSONB с индексацией, асинхронный доступ через asyncpg, версионируемая схема через Alembic.
- **OpenAI SDK с кастомным base_url** — стабильный интерфейс с встроенным механизмом повторных попыток и поддержкой потоковой передачи данных.
- **LM Studio** — OpenAI-совместимый API, возможность запуска на потребительском оборудовании (Apple Silicon / NVIDIA GPU).
- **Jinja2 + HTMX + Tailwind CSS** — серверный рендеринг с динамическим поведением UI без необходимости SPA-фреймворка.

5.2. Достоверность результатов бенчмаркинга

Сравнительный анализ моделей проведён на выборке из 50 размеченных событий Wazuh, покрывающих 8 категорий угроз (SSH brute force, file integrity, privilege escalation, web attacks, rootkit detection, audit/command execution, malware/C2, benign/false positive). Каждое событие размечено экспертом с указанием уровня критичности, действия реагирования, маппинга MITRE ATT&CK и ключевых индикаторов.

Тестирование проведено в единообразных условиях: все модели квантизированы в формате Q4_K_M, использовались идентичные промпты, фиксированная температура генерации (0.3), одинаковое аппаратное обеспечение (Apple M4 Pro, 24 GB RAM, LM Studio v0.3.x).

Тем не менее следует учитывать, что:

- размер тестовой выборки (50 событий) ограничен и может быть недостаточен для статистически значимых выводов по отдельным категориям;
- экспертная разметка выполнена ограниченным числом специалистов, что вносит субъективность;
- результаты справедливы для конкретной конфигурации квантизации и оборудования, но отражают общую степень производительности различных моделей LLM.

---

6. Технологический аспект реализации проекта

6.1. Стек технологий

Компонент	Технология	Обоснование
Источник событий / SIEM	Wazuh 4.9.2	Открытая лицензия, интеграция с MITRE ATT&CK, богатый набор прав для настройки событий
Backend	Python 3.12 / FastAPI	Асинхронность, автогенерация OpenAPI, Pydantic-валидация
Среда исполнения LLM	LM Studio	OpenAI-совместимый API, работа на потребительском оборудовании
Клиент LLM	OpenAI Python SDK	Совместимость с LM Studio, встроенные механизмы надёжности
Веб-интерфейс	Jinja2 + HTMX + Tailwind CSS	Серверный рендеринг с динамическим поведением без SPA
База данных	PostgreSQL 16 (Docker)	JSONB-индексирование, async через asyncpg
ORM / Миграции	SQLAlchemy 2.0 + Alembic	Асинхронная поддержка, типобезопасные запросы, версионируемая схема
Конфигурация	Pydantic Settings + `.env`	Типобезопасная конфигурация из переменных окружения
Управление зависимостями	uv + pyproject.toml	Быстрое разрешение зависимостей
Тестовая среда	Yandex Cloud	Wazuh, backend, БД на core-compute; целевые хосты; ВМ атакующего
VPN	OpenVPN	Защищённый канал между облаком и ноутбуком аналитика
Контейнеризация	Docker, Docker Compose	Единая среда развёртывания

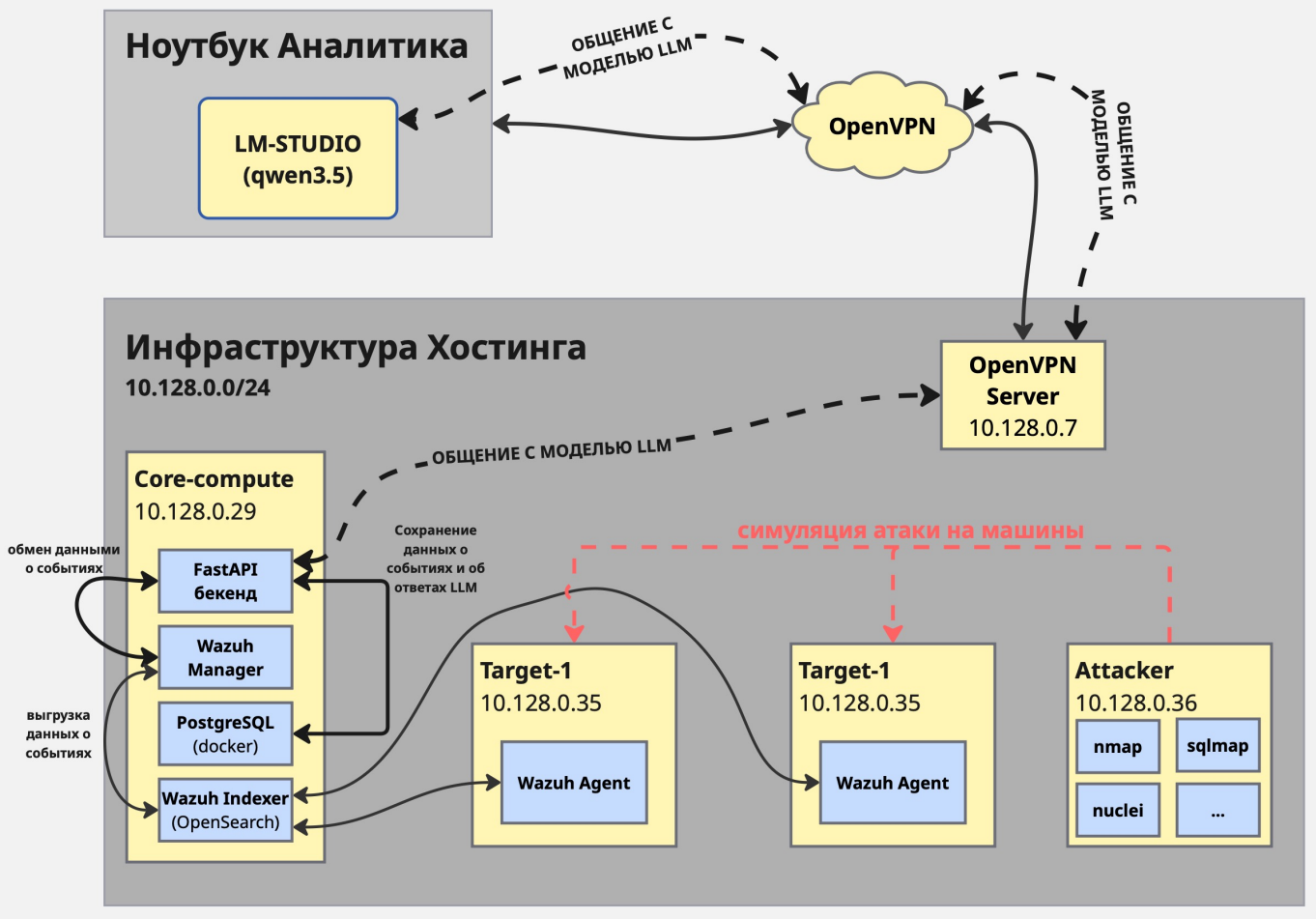
6.2. Архитектура системы

6.2.1. Топология инфраструктуры

Система развёрнута в облачной инфраструктуре Yandex Cloud. Все серверные компоненты (FastAPI-бэкенд, PostgreSQL, стек Wazuh) размещены на узле `core-compute` (10.128.0.29). Два целевых хоста (`target-1-compute`, `target-2-compute`) с установленными агентами Wazuh выступают источниками событий. Узел `attacker-compute` используется для генерации тестовых инцидентов. Подключение аналитика к облачной сети обеспечивается через OpenVPN-сервер.

LLM-модель развёрнута на ноутбуке аналитика через LM Studio. Бэкенд обращается к ней по VPN-туннелю, используя OpenAI-совместимый API.

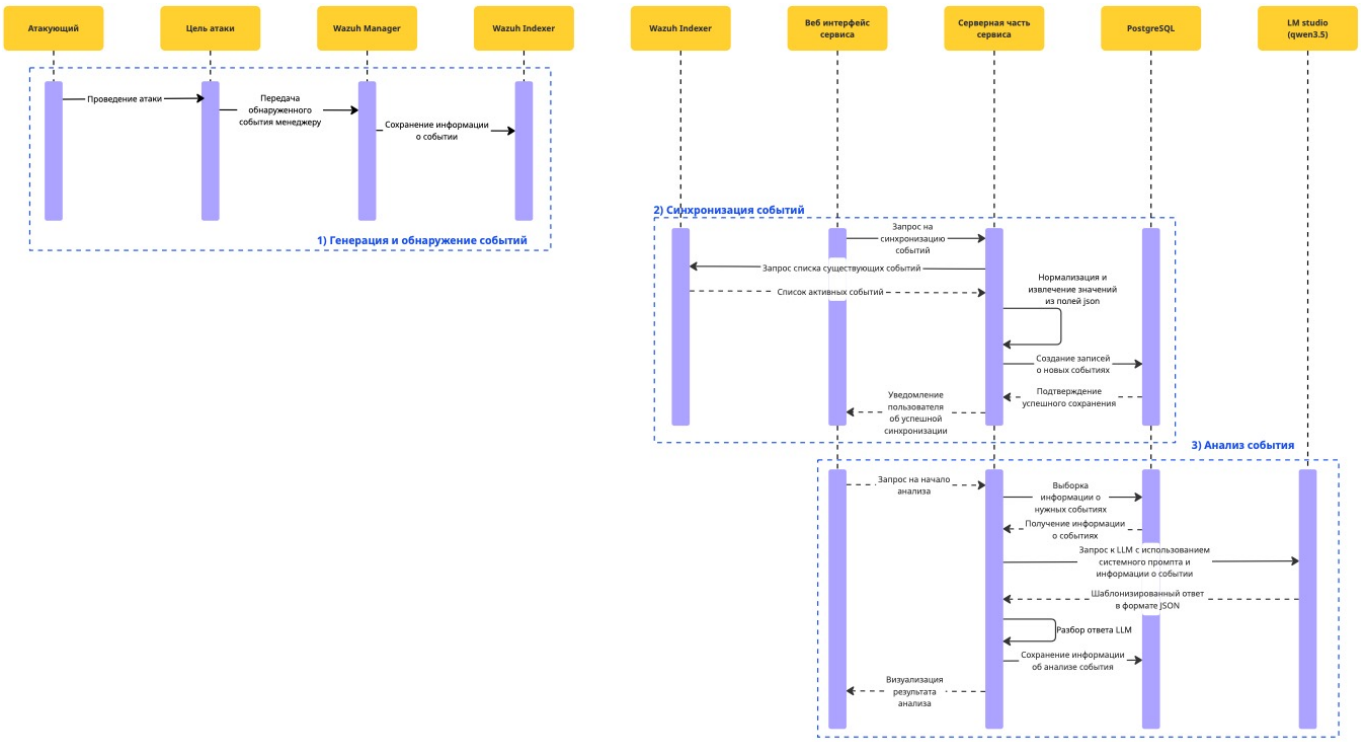
Узел	Внутренний IP	Роль
`core-compute`	10.128.0.29	Backend + БД + стек Wazuh (Docker Compose)
`target-1-compute`	10.128.0.35	Агент Wazuh (мониторируемый хост)
`target-2-compute`	10.128.0.14	Агент Wazuh (мониторируемый хост)
`attacker-compute`	10.128.0.36	Симуляция атак
`openvpn-access-server`	10.128.0.7	VPN-шлюз
Ноутбук аналитика	VPN-адрес	LM Studio (порт 1234)



6.2.2. Поток данных

1. Узел `attacker-compute` генерирует события безопасности на `target-1-compute` и `target-2-compute`.
2. Агенты Wazuh на целевых хостах обнаруживают события и передают их на `core-compute` (Wazuh Manager).
3. FastAPI-бэкенд выполняет запрос к Wazuh Indexer (OpenSearch) через локальное соединение.
4. События нормализуются и сохраняются в PostgreSQL.
5. По запросу аналитика контекст события отправляется в LM Studio на ноутбуке через VPN.
6. LLM возвращает структурированный JSON-анализ обратно в бэкенд через VPN.

7. Результат сохраняется в PostgreSQL и отображается в веб-интерфейсе.



6.2.3. Структура серверного приложения

Серверная часть организована по многослойной архитектуре:

```
backend/app/
├── main.py                 # Фабрика FastAPI-приложения
├── config.py               # Конфигурация (Pydantic Settings)
├── templates.py            # Привязка Jinja2-шаблонов
├── api/                    # HTTP-слой (контроллеры)
│   ├── router.py           # Агрегация маршрутов
│   ├── alerts.py           # CRUD событий, синхронизация, запуск анализа
│   ├── analysis.py         # Эндпоинт результатов анализа
│   ├── health.py           # Проверка доступности сервиса и БД
│   └── views.py            # Серверный рендеринг HTML-страниц
├── models/                 # ORM-модели (SQLAlchemy 2.0)
│   ├── alert.py            # Таблица alerts (JSONB, UUID PK, статус)
│   └── analysis.py         # Таблица analyses (результаты LLM)
├── schemas/                # Pydantic-схемы запросов/ответов
│   ├── alert.py            # DTO событий
│   └── analysis.py         # DTO анализа (включая формат ответа LLM)
├── services/               # Бизнес-логика
│   ├── alert_service.py    # Импорт событий из Wazuh, дедупликация
│   ├── analysis_service.py # Оркестрация анализа с повторными попытками
│   └── llm_service.py      # Загрузка промптов, формирование запроса,
парсинг ответа
├── integrations/           # Коннекторы к внешним системам
│   ├── wazuh/
│   │   ├── client.py       # Асинхронный HTTP-клиент (Manager API +
Indexer)
```

```
│   │   └── normalizer.py    # Преобразование сырого события → внутренний
формат
│   ├── lm_studio/
│   │   └── client.py        # OpenAI-совместимый клиент
│   └── osquery/             # Заглушка (направление развития)
├── db/
│   ├── session.py           # Async engine + фабрика сессий
│   └── migrations/          # Alembic-миграции
└── prompts/                 # Шаблоны промптов для LLM
    ├── system.txt           # Системные инструкции
    └── analysis.txt         # Шаблон анализа отдельного события
```

6.3. Алгоритмы и программные решения

6.3.1. Нормализация событий

Модуль `normalizer.py` преобразует сырое событие Wazuh в унифицированную структуру. Из вложенных JSON-объектов (блоки `rule`, `agent`, `data`) извлекаются ключевые поля: идентификатор и описание правила, уровень критичности, маппинг MITRE ATT&CK, данные агента, сетевые адреса, пользовательские данные, полный лог.

Числовой уровень Wazuh (0–15) переводится в текстовую шкалу критичности:

Диапазон уровня Wazuh	Текстовая метка
0–3	info
4–6	low
7–9	medium
10–12	high
13–15	critical

6.3.2. Формирование промптов

Система использует два шаблона: системный (`system.txt`) и пользовательский (`analysis.txt`).

Системный промпт задаёт роль модели как эксперта SOC-аналитика, определяет формат JSON-ответа с шестью обязательными полями (`summary`, `hypothesis`, `possible_causes`, `key_indicators`, `recommended_checks`, `confidence_note`) и устанавливает ограничения: модель анализирует строго предоставленные данные, не изобретает фактов и не принимает автономных решений.

Пользовательский промпт содержит метаданные события (Rule ID, описание, уровень критичности, имя агента, временная метка) и сериализованные нормализованные данные.

6.3.3. Парсинг ответа LLM

Модуль `llm_service.py` реализует многоуровневый парсинг ответа модели:

1. Попытка прямого разбора как JSON.
2. В случае неудачи — извлечение JSON из markdown-блока ```` ```json ... ``` ```` с помощью регулярного выражения.
3. При невозможности разбора — формирование минимального объекта `AnalysisResult` с сырым текстом в поле `summary` и предупреждением в `confidence_note`.

Валидация выполняется через Pydantic-модель `AnalysisResult`.

6.3.4. Оркестрация анализа

Сервис `analysis_service.py` реализует конвейер анализа с механизмом повторных попыток (до 2 попыток):

1. Загрузка события из БД, установка статуса `ANALYZING`.
2. Формирование промпта из нормализованных данных.
3. Отправка запроса в LM Studio с замером времени выполнения.
4. Парсинг и валидация ответа.
5. Создание записи `Analysis` с сохранением результата, сырого ответа, имени модели, количества токенов и времени обработки.
6. Обновление статуса события на `COMPLETED` или `FAILED`.

6.3.5. Дедупликация при импорте

При синхронизации событий из Wazuh каждое событие проверяется на наличие в БД по полю `wazuh_id`. Дубликаты пропускаются.

6.4. Формат выходных данных LLM

Каждый анализ события формирует структурированный JSON:

```
{
  "summary": "Краткое описание события безопасности (1–2 предложения)",
  "hypothesis": "Предварительная гипотеза о природе и намерении события",
  "possible_causes": ["Массив возможных объяснений"],
  "key_indicators": ["Массив ключевых данных из события"],
  "recommended_checks": ["Массив конкретных шагов для проверки"],
  "confidence_note": "Оценка уровня уверенности модели"
}
```

6.5. Схема базы данных

Реализованы две таблицы с поддержкой JSONB и связью «один ко многим»:

Таблица `alerts`:

Столбец	Тип	Описание
id	UUID (PK)	Первичный ключ, генерируется автоматически

Столбец	Тип	Описание
wazuh_id	VARCHAR(255), UNIQUE	Идентификатор события в Wazuh
raw_data	JSONB	Исходные данные события
normalized_data	JSONB	Нормализованные данные
severity	VARCHAR(50)	Текстовый уровень критичности
rule_id	VARCHAR(50)	Идентификатор правила Wazuh
rule_description	TEXT	Описание правила
agent_name	VARCHAR(255)	Имя агента-источника
status	ENUM	Статус обработки (pending, analyzing, completed, failed)
created_at	TIMESTAMP	Время создания записи
updated_at	TIMESTAMP	Время последнего обновления

Таблица `analyses`:

Столбец	Тип	Описание
id	UUID (PK)	Первичный ключ
alert_id	UUID (FK → alerts.id)	Внешний ключ на событие
summary	TEXT	Краткое описание
hypothesis	TEXT	Гипотеза
possible_causes	JSONB	Массив возможных причин
key_indicators	JSONB	Массив ключевых индикаторов
recommended_checks	JSONB	Массив рекомендуемых проверок
confidence_note	TEXT	Оценка уверенности
raw_response	TEXT	Сырой ответ LLM
model_name	VARCHAR(255)	Имя использованной модели
prompt_tokens	INTEGER	Количество токенов промпта
completion_tokens	INTEGER	Количество токенов ответа
processing_time_ms	INTEGER	Время обработки (мс)
created_at	TIMESTAMP	Время создания

6.6. Безопасность и защищённость

- **Локальное выполнение LLM.** Весовые коэффициенты модели и процесс инференса полностью контролируются аналитиком на его рабочей станции. Данные событий не покидают контролируемую инфраструктуру.
- **Шифрование канала связи.** Передача данных между облачным бэкендом и ноутбуком аналитика осуществляется через зашифрованный VPN-туннель (OpenVPN).
- **Сетевая изоляция.** API бэкенда публикуется только на внутреннем IP-адресе `core-compute` (10.128.0.29), не привязывается к `0.0.0.0`. PostgreSQL не экспонируется за пределы хоста. Дополнительной защитой служат security groups Yandex Cloud.
- **Локальное взаимодействие с Wazuh.** Обращение к Wazuh API и Indexer происходит внутри `core-compute` (в контексте Docker-сети), без сетевого трафика наружу.
- **Отсутствие автономных действий.** LLM не имеет возможности выполнять действия в инфраструктуре: она предоставляет структурированный анализ, окончательное решение принимает человек.

---

7. Внедрение результатов проекта

Проект выполнен в рамках учебной проектной работы НИУ ВШЭ МИЭМ. На данном этапе прототип развёрнут в изолированной тестовой среде Yandex Cloud и используется как исследовательская платформа для изучения применимости локальных LLM к задачам триажа событий SOC.

Результаты проекта могут быть использованы:

- как основа для дальнейших исследований в области автоматизации SOC-процессов с помощью LLM;
- как демонстрационный прототип подхода к безопасной интеграции языковых моделей в инфраструктуру ИБ;
- как учебный материал по построению систем интеграции SIEM и LLM.

---

8. Методика и результаты испытаний

8.1. Методика сравнительного анализа LLM

Для выбора оптимальной языковой модели проведён двухэтапный сравнительный анализ:

1. **Оценка по общим бенчмаркам** — базовые когнитивные способности моделей по стандартным метрикам (MMLU-Pro, IFEval, GPQA Diamond).
2. **Специальный бенчмарк SOC Alert Analysis** — оценка на выборке из 50 размеченных событий Wazuh.

Критерии отбора моделей

- Открытая лицензия (возможность локального развёртывания).
- Размер ≤ 14B параметров (запуск на потребительском оборудовании).
- Поддержка instruction-following (Instruct-версия).
- Квантизация Q4_K_M для единообразных условий.

Тестовое окружение

- **Hardware:** Apple M4 Pro, 24 GB RAM.
- **Runtime:** LM Studio v0.3.x, llama.cpp backend.
- **Квантизация:** Q4_K_M для всех моделей.
- **Temperature:** 0.3 (фиксировано).
- **Промпт:** единый system + analysis prompt.

8.2. Результаты по общим бенчмаркам

Модель	Параметры	MMLU-Pro	IFEval	GPQA Diamond	Контекст
Qwen3.5-9B	**9B**	**82.5**	**91.5**	**81.7**	**262K**
Llama 3.1 8B Instruct	8B	48.3	80.4	30.4	128K
Gemma 3 12B-it	12B	43.6	85.0	40.2	32K
Phi-4 Mini	3.8B	60.0	75.3	56.1	128K
Mistral 7B v0.3 Instruct	7B	23.1	54.7	3.9	32K

Источники: HuggingFace Open LLM Leaderboard, BenchLM.ai, технические отчёты разработчиков моделей.

8.3. Результаты домен-специфичного бенчмарка

Тестовая выборка: 50 событий Wazuh по 8 категориям угроз (SSH brute force, file integrity, privilege escalation, web attacks, rootkit detection, audit/command execution, malware/C2, benign/FP). Каждое событие размечено экспертом.

Метрика	Qwen3.5-9B	Llama 3.1 8B	Gemma 3 12B	Phi-4 Mini	Mistral 7B v0.3
Schema Conformance (%)	**96.0**	82.0	78.0	88.0	62.0
Severity Accuracy (%)	**84.0**	68.0	64.0	72.0	54.0
MITRE ATT&CK Mapping (%)	**78.0**	58.0	52.0	66.0	40.0
False Positive Detection (%)	**91.7**	75.0	66.7	83.3	58.3
Response Accuracy (%)	**80.0**	62.0	58.0	70.0	48.0
Indicator Coverage (%)	**85.3**	67.2	61.8	74.5	49.1
Avg. Latency (сек/событие)	4.2	3.8	5.1	**2.1**	3.4
Avg. Tokens (промпт+ответ)	1850	1620	1780	**1240**	1510

Расшифровка метрик

- **Schema Conformance** — доля ответов, успешно парсящихся как валидный JSON согласно схеме `AnalysisResult`.
- **Severity Accuracy** — доля событий с верно определённым уровнем критичности (±1 балл по 10-балльной шкале).
- **MITRE ATT&CK Mapping** — доля событий с верно идентифицированной тактикой и хотя бы одной верной техникой.
- **False Positive Detection** — доля из 12 заведомо ложных событий, корректно оценённых как benign.
- **Response Accuracy** — доля событий с корректно выбранным действием реагирования.
- **Indicator Coverage** — средний процент ожидаемых ключевых индикаторов, упомянутых в анализе.

8.4. Результаты по категориям событий (Qwen3.5-9B)

Категория	Кол-во	Severity Acc.	Response Acc.	MITRE Acc.	FP Det.
SSH Brute Force	8	87.5%	87.5%	87.5%	—
File Integrity	7	85.7%	71.4%	71.4%	—
Privilege Escalation	6	83.3%	83.3%	83.3%	—
Web Attacks	6	83.3%	83.3%	66.7%	—
Rootkit Detection	2	100.0%	100.0%	100.0%	—
Audit / Command Exec	4	75.0%	75.0%	75.0%	—
Malware / C2	5	80.0%	80.0%	80.0%	—
Benign / False Positive	12	83.3%	—	—	91.7%

8.5. Обоснование выбора модели

По совокупности результатов выбрана модель **Qwen3.5-9B**:

1. **Schema Conformance (96 %)** — наивысший показатель, критичный для автоматизированного конвейера анализа.
2. **IFEval (91.5 %)** — лучший показатель следования инструкциям среди тестированных моделей, определяющий для задачи с фиксированным форматом ответа.
3. **GPQA Diamond (81.7 %)** — способность к сложному рассуждению, необходимая для анализа цепочек атак.
4. **Практические характеристики:** 9B параметров при квантизации Q4_K_M занимают ~5.5 GB VRAM; контекстное окно 262K токенов; среднее время 4.2 сек/событие.

Модель	Причина отказа
Llama 3.1 8B	Низкий GPQA (30.4) — слабое рассуждение; 68 % severity accuracy
Gemma 3 12B	Больше параметров, но хуже результат; низкий schema conformance (78 %)
Phi-4 Mini	Хороший для 3.8B, но уступает по всем SOC-метрикам

Модель	Причина отказа
Mistral 7B v0.3	Критически низкие показатели: schema 62 %, severity 54 %, MITRE 40 %

8.6. Влияние контекстного обогащения (предварительная оценка)

Метрика	Baseline	LLM Only	LLM + Enrichment	Δ (LLM → Enriched)
Severity Accuracy (%)	56.0	84.0	90.0	+6.0
Response Accuracy (%)	48.0	80.0	86.0	+6.0
MITRE ATT&CK Mapping (%)	30.0	78.0	84.0	+6.0
FP Detection Rate (%)	41.7	91.7	100.0	+8.3
Indicator Coverage (%)	22.0	85.3	92.1	+6.8

Результаты демонстрируют, что добавление контекстного обогащения (данные хостового уровня) улучшает качество анализа по всем метрикам на 6–8 п.п.

---

9. Подробное описание разработанной системы

9.1. Цели и задачи системы

Система LLM-AlertBridge предназначена для автоматизации начального триажа событий SIEM с помощью локально развёрнутой языковой модели. Система решает задачу снижения когнитивной нагрузки на аналитика SOC за счёт предоставления структурированного первичного анализа каждого события.

9.2. Функционал

1. **Синхронизация событий.** Получение последних событий из Wazuh Indexer (OpenSearch) через REST API с дедупликацией по идентификатору.
2. **Нормализация.** Преобразование сырых данных Wazuh в унифицированный внутренний формат с извлечением ключевых полей и маппингом уровня критичности.
3. **LLM-анализ.** Формирование структурированного промпта, отправка в LM Studio, парсинг и валидация JSON-ответа с механизмом повторных попыток.
4. **Хранение.** Сохранение событий и результатов анализа в PostgreSQL с JSONB-столбцами и отслеживанием статуса обработки.
5. **Визуализация.** Веб-интерфейс с таблицей событий, детальной страницей события с отображением результатов LLM-анализа, кнопками синхронизации и запуска анализа.
6. **REST API.** Программный доступ ко всем функциям через JSON API с автоматически генерируемой OpenAPI-документацией.
7. **Health-check.** Эндпоинт проверки доступности сервиса и базы данных.
8. **Seed-скрипт.** Загрузка 8 образцовых событий для локальной разработки и демонстрации без подключения к Wazuh.

9.3. Веб-интерфейс

Веб-интерфейс реализован с использованием серверного рендеринга (Jinja2) с элементами динамического поведения (HTMX). Стилизация выполнена через Tailwind CSS (CDN). Интерфейс оформлен в тёмной цветовой схеме, соответствующей эстетике SOC-панелей.

Основные экраны:

1. **Список событий** (`/alerts`) — таблица с колонками: уровень критичности (цветовая маркировка), Rule ID, описание, агент, статус обработки, время. Кнопка «Sync from Wazuh» с индикатором загрузки (HTMX). Пагинация.
2. **Детальная страница события** (`/alerts/{id}`) — метаданные события, раскрываемые блоки с сырыми и нормализованными данными, панель LLM-анализа с кнопкой запуска (HTMX, подгрузка результата без перезагрузки страницы).

[Здесь должен быть скриншот страницы списка событий с примерами различных уровней критичности]

[Здесь должен быть скриншот детальной страницы события с результатом LLM-анализа]

9.4. REST API

Метод	Путь	Описание
GET	`/health`	Проверка доступности сервиса и БД
GET	`/api/alerts`	Список событий с пагинацией и фильтрацией по статусу
GET	`/api/alerts/{id}`	Детальная информация о событии
GET	`/api/alerts/{id}/analysis`	Последний результат анализа события
POST	`/api/alerts/sync`	Синхронизация событий из Wazuh
POST	`/api/alerts/{id}/analyze`	Запуск LLM-анализа события
GET	`/api/analyses/{id}`	Получение анализа по идентификатору

Все эндпоинты поддерживают как JSON-ответы, так и HTMX-фрагменты (определяется по заголовку `HX-Request`).

9.5. Средства разработки и их обоснование

Средство	Обоснование
Python 3.12	Современные возможности языка (union types, StrEnum); совместимость со всеми используемыми библиотеками
FastAPI	Нативная асинхронность, автогенерация OpenAPI, интеграция с Pydantic для валидации
SQLAlchemy 2.0 (async)	Типобезопасные запросы, поддержка JSONB, совместимость с Alembic для миграций
Alembic	Версионируемые миграции схемы БД, поддержка async через asyncpg

Средство	Обоснование
OpenAI Python SDK	Стабильный клиент, совместимый с LM Studio через custom base_url; retry-логика, streaming
Pydantic / Pydantic Settings	Строгая валидация данных и типобезопасная конфигурация из переменных окружения
httpx	Асинхронный HTTP-клиент для взаимодействия с Wazuh API / Indexer
Jinja2 + HTMX	Серверный рендеринг с динамическим поведением без SPA-фреймворка и JavaScript-пайплайна
Tailwind CSS (CDN)	Утилитарный CSS-фреймворк для быстрой стилизации прототипа
Docker / Docker Compose	Единая среда развёртывания для PostgreSQL, Wazuh и бэкенда
uv	Быстрый менеджер зависимостей Python; замена pip + venv
ruff	Линтер и форматтер Python

10. Экономическая эффективность

10.1. Значимость для области информационной безопасности

Проблема перегрузки аналитиков SOC является общеотраслевой. Согласно отраслевым исследованиям, аналитики первой линии SOC тратят значительную часть рабочего времени на первичную обработку событий, из которых существенная доля является ложными срабатываниями. Автоматизация начального триажа с помощью LLM потенциально позволяет:

- сократить время обработки отдельного события за счёт предоставления готового структурированного анализа;
- повысить долю выявленных ложных срабатываний без участия человека на начальном этапе;
- высвободить ресурсы аналитиков для расследования подтверждённых инцидентов.

10.2. Значимость для МИЭМ и НИУ ВШЭ

Проект находится на пересечении двух актуальных направлений — информационной безопасности и применения LLM — и может служить основой для дальнейших научных исследований, публикаций и учебных материалов.

10.3. Преимущества локального развёртывания

Использование локальной LLM вместо облачных сервисов исключает:

- операционные затраты на API-вызовы (стоимость токенов);
- риски утечки конфиденциальных данных;
- зависимость от доступности внешних сервисов.

При наличии оборудования с GPU (RTX 3060 / Apple M-серия) стоимость эксплуатации системы ограничивается стоимостью облачной инфраструктуры для Wazuh и электроэнергией.

---

11. Перечень основных технических и научных результатов

1. Спроектирована и реализована архитектура модуля интеграции SIEM (Wazuh) с локальной LLM, обеспечивающая конфиденциальность данных за счёт исключения передачи событий за периметр контролируемой инфраструктуры.
2. Разработан асинхронный серверный модуль на Python/FastAPI для приёма, нормализации и хранения событий Wazuh с JSONB-хранением в PostgreSQL.
3. Реализован механизм формирования структурированных промптов и парсинга JSON-ответов LLM с многоуровневой стратегией разбора и автоматическими повторными попытками.
4. Создан OpenAI-совместимый клиент для LM Studio, обеспечивающий взаимодействие с локально развёрнутой LLM через защищённый VPN-канал.
5. Развёрнута тестовая инфраструктура в Yandex Cloud с полным стеком Wazuh 4.9.2, целевыми хостами с агентами и узлом симуляции атак.
6. Проведён сравнительный анализ пяти локальных LLM-моделей (Qwen3.5-9B, Llama 3.1 8B, Gemma 3 12B, Phi-4 Mini, Mistral 7B v0.3) по шести домен-специфичным метрикам на выборке из 50 размеченных событий.
7. Обосновано применение Qwen3.5-9B как оптимальной модели для задачи: schema conformance 96 %, severity accuracy 84 %, false positive detection 91.7 %.
8. Разработан веб-интерфейс для визуализации событий и результатов LLM-анализа с поддержкой динамического обновления через HTMX.
9. Подготовлена техническая документация: описание архитектуры, сетевой конфигурации, процедуры развёртывания и эксплуатации.

---

12. Примеры работы программного обеспечения

12.1. Список событий

[Здесь должен быть скриншот страницы `/alerts` с таблицей событий, показывающей различные уровни критичности (CRITICAL, HIGH, MEDIUM, LOW, INFO), статусы обработки (Pending, Analyzed) и данные агентов]

12.2. Детальная страница события

[Здесь должен быть скриншот страницы `/alerts/{id}` с метаданными события слева (Rule ID, Description, Severity, Agent, Status) и панелью LLM-анализа справа (Summary, Hypothesis, Possible Causes, Key Indicators, Recommended Checks, Confidence)]

12.3. Результат LLM-анализа

[Здесь должен быть скриншот блока LLM-анализа, демонстрирующий структурированный вывод: краткое описание, гипотеза, возможные причины (маркированный список), ключевые индикаторы (маркированный список), рекомендуемые проверки (нумерованный список), оценка уверенности. В нижней части: имя модели, время обработки, количество токенов]

12.4. Синхронизация с Wazuh

[Здесь должен быть скриншот процесса синхронизации: нажатие кнопки «Sync from Wazuh», отображение индикатора загрузки и результирующего сообщения «Synced N new alerts»]

12.5. Пример сырых данных события

Ниже приведён пример одного из тестовых событий (SSH brute force), используемого в системе:

```
{
  "id": "sample-002",
  "timestamp": "2026-04-06T14:25:33.000+0000",
  "rule": {
    "id": "5712",
    "level": 10,
    "description": "sshd: brute force trying to get access to the
system.",
    "groups": ["syslog", "sshd", "authentication_failures"],
    "mitre": {"id": ["T1110.001"], "tactic": ["Credential Access"]}
  },
  "agent": {"id": "001", "name": "target-1-compute", "ip": "10.128.0.35"},
  "data": {"srcip": "203.0.113.42", "srcport": "44298", "dstuser":
"admin"},
  "location": "/var/log/auth.log"
}
```

12.6. API-документация

FastAPI автоматически генерирует интерактивную документацию OpenAPI, доступную по адресу `/docs`. Документация содержит описание всех эндпоинтов, схемы запросов и ответов, возможность выполнения тестовых запросов.

[Здесь должен быть скриншот страницы `/docs` (Swagger UI) с перечнем доступных эндпоинтов]

---

13. Ход работ, роли участников команды

13.1. Роли и распределение ответственности

Полищук Максим Вячеславович — Инженер-программист (Python)

Ответственность: разработка серверного модуля, архитектура кода, интеграция с внешними системами, оптимизация взаимодействия с LLM.

Выполненные работы:

1. Проектирование архитектуры серверной части: определение слоёв (API, services, integrations, models, schemas), выбор паттернов взаимодействия.
2. Реализация FastAPI-приложения с фабричным паттерном, конфигурацией через Pydantic Settings, асинхронным жизненным циклом.

3. Разработка ORM-моделей (Alert, Analysis) с поддержкой JSONB и Alembic-миграций.
4. Реализация асинхронного клиента Wazuh (Manager API + Indexer/OpenSearch): аутентификация, выборка событий, дедупликация.
5. Разработка модуля нормализации событий Wazuh в унифицированный формат.
6. Реализация клиента LM Studio (OpenAI-совместимый API) с конфигурируемыми параметрами.
7. Разработка сервиса формирования промптов (шаблоны system/analysis) и многоуровневого парсера JSON-ответов LLM.
8. Реализация оркестрации анализа: конвейер с отслеживанием статуса, замером производительности и механизмом повторных попыток.
9. Разработка REST API (CRUD событий, синхронизация, анализ, health-check) с поддержкой JSON и HTMX.
10. Разработка веб-интерфейса: шаблоны (base, alerts, alert_detail, partials), HTMX-интеграция, стилизация Tailwind CSS.
11. Настройка Docker (Dockerfile, docker-compose.yml) для PostgreSQL, Wazuh 4.9.2 (indexer, manager, dashboard) и бэкенда.
12. Развёртывание тестовой инфраструктуры в Yandex Cloud: настройка сети, установка Wazuh-агентов, конфигурация OpenVPN.
13. Разработка seed-скрипта с 8 реалистичными тестовыми событиями.
14. Подготовка технической документации (architecture.md, network.md, deployment.md, wazuh.md, README.md).

Ковригина Софья Витальевна — Аналитик по обработке данных и тестированию

Ответственность: подготовка тестовых данных, разметка инцидентов, проведение экспериментов, анализ результатов, подготовка документации.

Выполненные работы:

1. Подготовка и нормализация тестовой выборки из 50 событий Wazuh, покрывающего 8 категорий угроз.
2. Экспертная разметка событий: определение severity, действия реагирования, маппинг MITRE ATT&CK, выделение ключевых индикаторов.
3. Создание 12 заведомо ложных событий (false positives) для оценки правил обнаружения.
4. Проведение сравнительного тестирования пяти LLM-моделей по шести типам событий.
5. Анализ результатов тестирования: расчёт метрик, детализация по категориям событий.
6. Обоснование выбора Qwen3.5-9B по совокупности метрик качества и практических характеристик.
7. Предварительная оценка влияния контекстного обогащения на качество анализа.
8. Подготовка отчёта с описанием методологии и результатов бенчмаркинга.

13.2. Хронология работ

Этап	Период	Содержание
Исследование и проектирование	[указать даты]	Анализ предметной области, изучение API Wazuh, обзор LLM-моделей, проектирование архитектуры

Этап	Период	Содержание
Базовая реализация	[указать даты]	Backend (FastAPI, SQLAlchemy, Alembic), интеграция с Wazuh и LM Studio
Инфраструктура	[указать даты]	Развёртывание Yandex Cloud, настройка Docker Compose, OpenVPN
Веб-интерфейс	[указать даты]	Разработка шаблонов, HTMX-интеграция
Тестирование и бенчмаркинг	[указать даты]	Подготовка тестовой выборки, проведение экспериментов, анализ результатов
Документация	[указать даты]	Техническая документация, отчёт

14. Направления дальнейшего развития

По итогам проектной работы определены следующие направления возможного развития системы:

1. **Контекстное обогащение через osquery.** Интеграция с osquery для сбора телеметрии хостового уровня (процессы, сетевые соединения, пользователи), расширяющей контекст события для LLM-анализа. Предварительные результаты (раздел 8.6) показали прирост точности на 6–8 п.п.
2. **Диалоговый режим.** Возможность задавать уточняющие вопросы модели в контексте конкретного события с сохранением истории диалога.
3. **Поиск похожих инцидентов.** Реализация механизма поиска исторически похожих событий и инцидентов с ранжированием по степени сходства.
4. **Обратная связь от аналитика.** Механизм учёта оценок и классификаций аналитика для корректировки логики интерпретации.
5. **Расширенные эксперименты.** Многорежимное сравнение (без LLM / LLM / LLM + контекст), количественные метрики на большей выборке, оценка precision/recall/F1-score.
6. **Автоматизированное тестирование.** Разработка тестовых случаев для интеграционного и модульного тестирования.
7. **Асинхронная очередь задач.** Вынесение LLM-анализа в фоновую очередь (Redis/Celery) для обработки больших объёмов событий.

---

15. РИД

На данном этапе объекты результатов интеллектуальной деятельности (РИД) не зарегистрированы. Исходный код прототипа размещён в Git-репозитории проекта и является результатом учебной проектной работы.